



UNIVERSIDADE ESTADUAL DE CAMPINAS
INSTITUTO DE MATEMÁTICA, ESTATÍSTICA E COMPUTAÇÃO CIENTÍFICA
DEPARTAMENTO DE MATEMÁTICA APLICADA



Felipe Carisio Hirt Ferreira Romano Roza

O problema de corte irregular: uma abordagem via discretização

Campinas
25/06/2025

Felipe Carisio Hirt Ferreira Romano Roza

O problema de corte irregular: uma abordagem via discretização¹

Monografia apresentada ao Instituto de Matemática, Estatística e Computação Científica da Universidade Estadual de Campinas como parte dos requisitos para obtenção de créditos na disciplina Projeto Supervisionado I, sob a orientação da Profa. Kelly Cristina Poldi.

¹Este trabalho foi financiado pelo PIBIC/UNICAMP, cota 2024-2025.

Resumo

Este projeto estuda o Problema de Corte Irregular, amplamente referido como *nesting*. As aplicações práticas deste problema são diversas, como corte de moldes na indústria têxtil, indústria de couro e corte de placas metálicas da indústria automotiva; porém, existem aplicações em múltiplas áreas que envolvem cortes e/ou empacotamento de itens de irregulares. O projeto estuda e desenvolve uma modelagem matemática para resolução do problema de corte irregular via programação linear inteira. Para implementação e testes computacionais, usamos a linguagem Julia, com a biblioteca JuMP e os *solvers* CPLEX e Gurobi.

Abstract

This project studies the Irregular Cutting Problem, widely referred to as nesting. Practical applications of such problem are diverse, such as cutting patterns in the textile industry, leather industry, and cutting metal plates in the automotive industry. However, applications exist in multiple areas that involve the cutting and/or packing of irregular items. The project involves the study and development of a mathematical model which is solved via integer linear programming. For computational implementation and testing, we use the Julia language with the JuMP library, and the solvers CPLEX and Gurobi.

Sumário

1	Introdução	6
2	Descrição do problema	7
2.1	Representação das peças	7
2.2	<i>Inner-Fit Polygon</i>	8
2.3	<i>No-Fit Polygon</i>	8
2.4	Modelagem matemática	10
3	Desenvolvimento	11
3.1	Implementação computacional	11
3.1.1	Métodos auxiliares	11
3.1.2	Modelagem com JuMP	12
3.1.3	Função <i>plotting</i>	14
3.2	Resultados dos testes computacionais preliminares	14
3.2.1	Primeiro exemplo	14
3.2.2	Segundo exemplo	14
3.2.3	Terceiro exemplo	15
3.3	Dificuldades, correções e melhorias	16
4	Resultados finais	17
4.1	Conjunto BLAZ	17
4.2	Conjunto SHAPES	19
4.3	Conjunto SHIRTS	21
5	Conclusão	22

1 Introdução

O estudo dos problemas de otimização de corte é um tópico de grande interesse na comunidade acadêmica. As primeiras pesquisas, focadas no chamado *cutting stock problem* (problema de corte de estoque), datam da década de 1960 (Kantorovich [8], Gilmore e Gomory [6, 5]). O objetivo é otimizar o corte de peças maiores (objetos) em peças menores (itens) de forma a minimizar a perda de material, por exemplo. Inicialmente, os itens considerados eram retângulos. Aproximadamente duas décadas depois, o foco da comunidade científica começou a se expandir para incluir também o desafio de otimizar o corte de itens de formato irregular, conhecido na literatura também como problema de *nesting*. A evolução e a classificação desses problemas são detalhadas em trabalhos subsequentes, como na tipologia de Wäscher *et al.* [10].

O problema de *nesting* (ou corte irregular) é recorrente em contextos práticos diversos, particularmente em setores industriais que lidam com materiais de geometria não uniforme. No entanto, sua modelagem matemática é complexa, tanto do ponto de vista geométrico quanto combinatório, o que resulta em uma literatura significativamente mais limitada quando comparada ao problema de corte regular.

Diferentemente do corte regular, em que tanto as peças quanto a área disponível para corte assumem formas geométricas simples (tipicamente retangulares), no problema de corte irregular as subdivisões a serem realizadas possuem contornos arbitrários. Além disso, a superfície original sobre a qual o corte será efetuado pode apresentar bordas irregulares ou regiões inutilizáveis, o que impõe restrições adicionais ao modelo.

Um exemplo representativo do problema de corte ocorre na indústria do vestuário. Quando se utiliza tecido, matéria-prima sobre a qual os itens são alocados — tipicamente um rolo de tecido — é considerado um objeto de geometria regular, sobre o qual se realizam alocações de peças com formatos irregulares. Esse cenário corresponde a uma classe de problemas de *nesting* em que o contorno do objeto base é regular, enquanto os itens a serem dispostos apresentam formas arbitrárias.

Por outro lado, aplicações envolvendo materiais como couro animal apresentam desafios adicionais. Nesse caso, o objeto base, além de possuir contorno irregular, pode conter regiões inutilizáveis devido a imperfeições naturais do material, o que exige uma modelagem mais sofisticada. Neste trabalho, concentramos a análise no primeiro cenário descrito: problemas de *nesting* em objetos de geometria regular com itens de formas irregulares.

A otimização do corte resulta não apenas em uma redução de custos, mas também fomenta práticas de produção mais sustentáveis. Em indústrias como a têxtil, naval, de calçados e metalúrgica, onde os materiais representam uma parte significativa do custo produtivo, quaisquer melhorias na eficiência do *layout* de corte podem gerar uma economia de recursos bastante relevante. Adicionalmente, ao impacto econômico direto, a resolução eficiente de problemas de *nesting* é um elemento fundamental para a automação de processos, contribuindo para o aumento da precisão e da velocidade de produção. Com isso, o número de pesquisas vêm crescendo nas últimas décadas, como exemplo, podemos citar Bennel [1] e [2], Cherri *et al.* [3], Gomes e Oliveira [7], Toledo *et al.* [9].

2 Descrição do problema

O problema abordado nesta monografia consiste em, dada uma área retangular, denominada *board*, e um conjunto de t peças distintas, de dimensões conhecidas, alocar essas peças no *board*. Inicialmente o *board* possui altura pré-definida e fixa, mas o seu comprimento é “infinito”. O objetivo é posicionar todas as peças de forma a minimizar o comprimento utilizado do board, ou seja, minimizar a área utilizada.

Na literatura, podemos encontrar algumas abordagens para o problema de *nesting*, que usam heurísticas ou por programação matemática. Nesse trabalho, abordamos o problema via discretização, utilizando como base o modelo de programação linear inteira definido em Toledo *et al.* [9].

Para representar o problema, fazemos uma discretização do *board*, ou seja, ele é dividido em um *grid*, formando assim um conjunto de pontos que são utilizados como referência para o posicionamento das peças. Vamos definir algumas constantes com relação ao *board*. Assim, temos os seguintes parâmetros:

L : comprimento do *board* (inicialmente um limitante superior);

W : altura do *board*;

g_x : resolução do *grid* no eixo x ;

g_y : resolução do *grid* no eixo y ;

C : número de colunas ($\lfloor L/g_x \rfloor + 1$);

R : número de linhas ($\lfloor W/g_y \rfloor + 1$);

D : total de pontos ($C \times R$).

A seguir, definimos alguns conjuntos relevantes:

$\mathcal{T} = \{1, \dots, T\}$: tipos de peças;

$\mathcal{C} = \{0, \dots, C - 1\}$: colunas;

$\mathcal{R} = \{0, \dots, R - 1\}$: linhas;

$\mathcal{D} = \{1, \dots, D\}$: pontos do *grid*.

2.1 Representação das peças

Cada peça tipo t é tratada como um polígono, representado por um conjunto de vértices que são definidos com relação a um vértice (ponto) de referência. Para definir uma peça tipo t , consideramos os seguintes parâmetros:

- $(0, 0)$: vértice de referência;
- $\langle (x_t^1, y_t^1), (x_t^2, y_t^2), \dots, (x_t^{v_t}, y_t^{v_t}) \rangle$: lista de vértices da peça tipo $t \in \mathcal{T}$, sentido horário;

- $x_t^m, x_t^M, y_t^m, y_t^M$: coordenadas do envelope retangular da peça tipo $t \in \mathcal{T}$,
$$\begin{cases} x_t^m = \min_{i \in \{1 \dots v_t\}} x_t^i; \\ x_t^M = \max_{i \in \{1 \dots v_t\}} x_t^i; \\ y_t^m = \min_{i \in \{1 \dots v_t\}} y_t^i; \\ y_t^M = \max_{i \in \{1 \dots v_t\}} y_t^i. \end{cases}$$

Além disso, também temos como parâmetro:

- q_t : o número de peças do tipo $t \in \mathcal{T}$ a serem posicionadas no *board*.

2.2 Inner-Fit Polygon

Uma restrição para todo problema de corte é que a peça deve estar totalmente posicionada dentro do objeto. Para garantir que a solução respeite essa restrição, em problemas de *nesting*, utilizamos o conceito de *Inner-Fit Polygon* ($\mathcal{IFP}$) (mais detalhes em Gomes e Oliveira [7]).

Dado um *board* e uma peça $t \in \mathcal{T}$, o $\mathcal{IFP}_t$ é o conjunto de todos os pontos $d \in \mathcal{D}$ nos quais a peça pode ser posicionada de forma a estar completamente dentro do *board*. Se o *board* for retangular, o $\mathcal{IFP}_t$ também será.

2.3 No-Fit Polygon

Outra restrição elementar para os problemas de *nesting* é que as peças não podem se sobrepor (*overlap*). Assim, para respeitar essa condição, utilizamos o conceito de *No-Fit Polygon* ($\mathcal{NFP}$).

Dado um par de peças $t, u \in \mathcal{T}$, o $\mathcal{NFP}_{t,u}^d$ é o conjunto de todos os pontos que, uma vez que a peça t está posicionada em d , caso a peça u seja posicionada, elas se sobreponham. Ou seja, o $\mathcal{NFP}_{t,u}^d$ restringe o posicionamento de peças posteriores.

O conjunto final que representa as possíveis posições da n -ésima peça a ser posicionada é dado por:

$$P_n = \mathcal{IFP}_n \setminus \bigcup_{i=1}^{n-1} \mathcal{NFP}_{i,n}^{d_i}$$

Para ilustrar os conceitos de $\mathcal{IFP}$ e $\mathcal{NFP}$, considere um *board* de comprimento 10 e altura 7. Considere também as quatro peças seguintes, ilustradas na Figura 1, a serem posicionadas:

1. Quadrado 3×3 ;
2. Losango com diagonais iguais a 4;
3. Triângulo de base (comprimento) 4 e altura 3;
4. Peça não convexa em formato de “u”, com 4 de comprimento e 2 de altura.

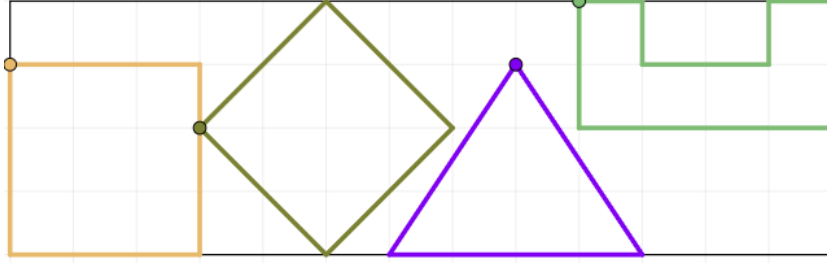


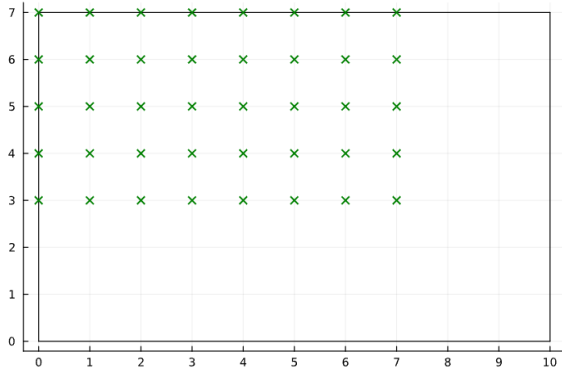
Figura 1: Ilustração de quatro peças a serem posicionadas em um *board* 10×7 .

Note que para cada peça há um vértice marcado; este é o vértice de referência (coordenada (0,0)) mencionado na Seção 2.1. A seguir, apresentamos um trecho de código de declaração dessas peças.

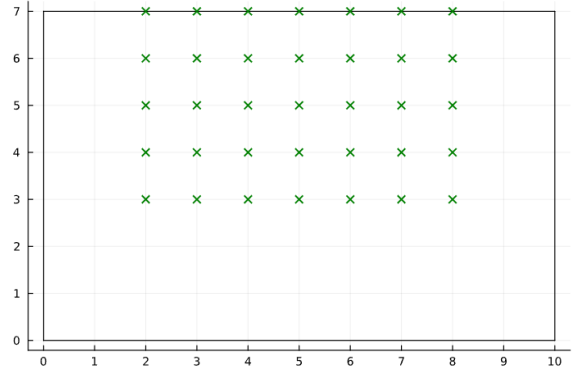
```

1  pecas = [
2      [(0.0,0.0), (3.0,0.0), (3.0,-3.0), (0.0,-3.0), (0.0,0.0)], #quadrado 3x3
3      [(0.0,0.0), (2.0,2.0), (4.0,0.0), (2.0,-2.0), (0.0,0.0)], #quadrado rotacionado
4      [(0.0,0.0), (2.0,-3.0), (-2.0,-3.0), (0.0,0.0)], #triangulo
5      [(0.0,0.0), (1.0,0.0), (1.0,-1.0), (3.0,-1.0), (3.0,0.0), (4.0,0.0), (4.0,-2.0),
        ↪ (0.0,-2.0), (0.0,0.0)], #forma nao convexa "u"
6  ]
7  q = [1, 1, 1, 1]

```



(a) IFP_1



(b) IFP_3

Figura 2: Representações de (a) IFP_1 e (b) IFP_3 .

Na Figura 2a, apresentamos o *Inner-Fit Polygon* para a peça 1 (IFP_1), ou seja, para respeitar a restrição da peça estar totalmente contida dentro do *board*, os pontos verdes ($\times$) representam as possíveis posições em que um quadrado 3×3 pode ser posicionado, considerando seu vértice de referência (canto superior esquerdo, identificado na Figura 1).

Na Figura 2b, temos a representação para o *Inner-Fit Polygon* da peça 3 (IFP_3), ou seja, do triângulo. Novamente, os pontos verdes ($\times$) representam os possíveis locais onde podem ser posicionadas uma peça tipo 3, garantindo que a peça esteja totalmente contida no *board*.

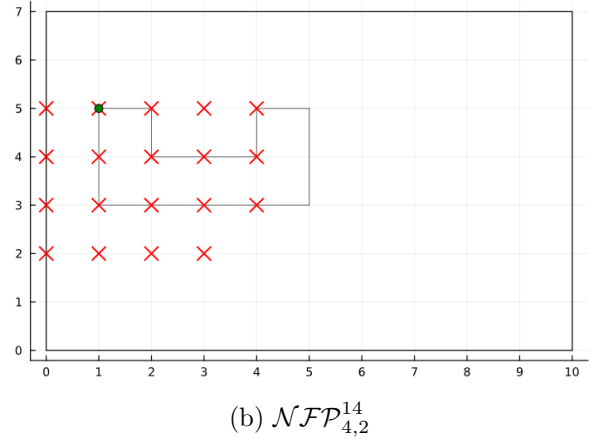
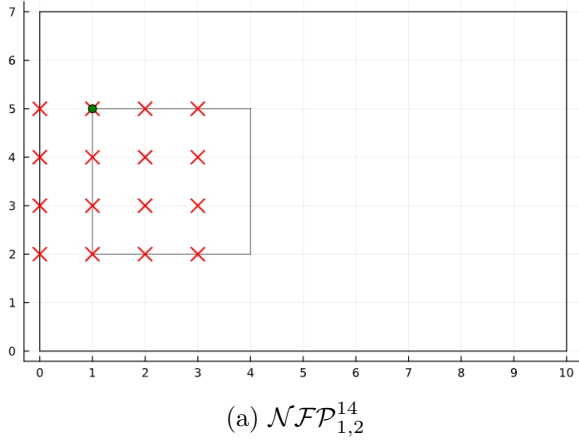


Figura 3: Representações de (a) $\mathcal{NFP}_{1,2}^{14}$ e (b) $\mathcal{NFP}_{4,2}^{14}$.

Na Figura 3a, apresentamos o *No-Fit Polygon* da peça 2 com relação à peça 1 no ponto 14 (destacado em verde), ou seja, os pontos vermelhos (×) marcam todas as posições nas quais um losango (peça 2) não pode ser posicionado dado que o quadrado (peça 1) já está no ponto 14. Na Figura 3b, apresentamos, equivalentemente, o *No-Fit Polygon* do polígono 2 com o polígono 4 posicionado no ponto 14 ($\mathcal{NFP}_{4,2}^{14}$), ou seja, os pontos vermelhos (×) marcam as posições nas quais o losango (peça 2) não pode ser posicionado dado que a peça 4 (tipo “u”) já está posicionada no ponto 14.

Vale ressaltar que o $\mathcal{NFP}$ apenas considera pontos válidos para o $\mathcal{IFP}$ daquela mesma peça. Por exemplo, no $\mathcal{NFP}_{1,2}^{14}$ fica claro que o losango não poderia ser posicionado no ponto (0, 6) pois causaria *overlap* com o quadrado, mas nessa posição a peça não estaria inteiramente dentro do *board*, não respeitando o $\mathcal{IFP}$, portanto é descartada.

2.4 Modelagem matemática

Nesta seção, apresentamos uma formulação matemática para o problema de alocação das peças (irregulares) em um *board*. Primeiramente, é importante lembrar que, na abordagem escolhida para esta modelagem, a altura do *board* (W) é fixa e pré-definida. Para o comprimento (L) é utilizado um valor grande o suficiente (limitante superior), de modo a garantir a factibilidade do problema. A partir disso, determina-se o menor valor possível de L que permita o posicionamento de todas as peças necessárias.

Considere os parâmetros definidos anteriormente e considere também as seguintes variáveis de decisão:

- $\delta_t^d = \begin{cases} 1, & \text{se temos uma peça do tipo } t \text{ no ponto } d, \text{ com } d \in \mathcal{D}, t \in \mathcal{T}; \\ 0, & \text{caso contrário;} \end{cases}$
- z o comprimento utilizado do *board*.

O modelo matemático, proposto em Toledo *et al.* [9], é dado por:

$$\min z \quad (1)$$

sujeito a:

$$(c \times g_x + x_t^M) \times \delta_t^d \leq z, \quad \forall d \in \mathcal{IFP}_t, \quad \forall t \in \mathcal{T} \quad (2)$$

$$\sum_{d \in \mathcal{IFP}_t} \delta_t^d = q_t, \quad \forall t \in \mathcal{T} \quad (3)$$

$$\delta_u^e + \delta_t^d \leq 1, \quad \forall e \in \mathcal{NFP}_{t,u}^d, \quad \forall t, u \in \mathcal{T}, \quad \forall d \in \mathcal{IFP}_t \quad (4)$$

$$\delta_t^d \in \{0, 1\}, \quad \forall d \in \mathcal{IFP}_t, \quad \forall t \in \mathcal{T} \quad (5)$$

$$z \in \mathbb{Z}^+ \quad (6)$$

A função objetivo (1) tem por objetivo minimizar o comprimento utilizado do *board*. O comprimento z é delimitado pela restrição (2). O conjunto de restrições (3) garante que todas as peças sejam posicionadas. A não sobreposição das peças é garantida pelas restrições (4). As restrições (5) e (6) definem os domínios das variáveis.

3 Desenvolvimento

3.1 Implementação computacional

A implementação computacional do modelo de programação linear inteira (1)-(5) foi feita em linguagem Julia, versão 1.11.2, com as bibliotecas JuMP e Plots, os *solvers* utilizados foram CPLEX v22.1.1 e Gurobi v12.0.0, em um computador munido de um *AMD Ryzen 5 7600x* operando em *4.7 GHz* e *32GB* de memória *RAM*. Todos os testes foram realizados completamente de maneira local, sem a utilização de servidores de qualquer espécie, garantindo, assim, maior consistência em seus resultados.

3.1.1 Métodos auxiliares

Nesta seção, estão apresentados os procedimentos para a resolução do problema de corte irregular usando a modelagem proposta.

Para o *Inner-Fit Polygon (IFP)* foi utilizada a implementação detalhada abaixo. Foi utilizado o conceito de envelope retangular, mencionado na Seção 2.1, para analisar todas as coordenadas e determinar em quais uma peça do tipo $t \in \mathcal{T}$ pode ser posicionada, permanecendo inteiramente dentro do *board* e respeitando a restrição.

```

1 IFP = Dict{Int, Vector{Int}}{Int}()
2
3 for (i, peca) in enumerate(pecas)
4     maxX = maximum(x for (x,y) in peca)
5     maxY = maximum(y for (x,y) in peca)
6     minX = minimum(x for (x,y) in peca)
7     minY = minimum(y for (x,y) in peca)
8

```

```

9      largura = maxX - minX
10     altura = maxY - minY
11
12     validos = []
13     for d in D
14         c = floor((d - 1) / R)
15         r = d - c * R - 1
16         if (c + maxX <= L) && (c + minX >= 0) && (r + maxY <= H) && (r + minY >= 0)
17             push!(validos, d) #convertendo a coordenada para indice de ponto
18         end
19     end
20     IFP[i] = validos
21 end

```

Já para a implementação do *No-Fit Polygon* ($\mathcal{NFP}$) utilizou-se uma abordagem mais elaborada. Observe que analisar os limites do envelope de cada tipo de peça, como no $\mathcal{IFP}$, não é suficiente. Como as peças tratadas são irregulares, suas intersecções podem não acontecer mesmo que os envelopes apresentem *overlapping*, portanto deve ser feita uma análise mais completa e detalhada.

Para isso, foi utilizada a biblioteca PolygonOps com a função `inpolygon(ponto, polígono)`, que analisa se o ponto está dentro do polígono, e foi realizada uma verificação ponto a ponto e lado a lado de uma peça e comparada à outra, esta já posicionada. Observe que esse processo deve ser realizado entre todos os tipos de peça, e em todas as suas possíveis posições no *board* ($\mathcal{IFP}$).

```

1  NFP = Dict{Tuple{Int, Int, Int}, Vector{Int}}{ }
2
3  for (i, peca1) in enumerate(pecas) #peça "que está posicionada"
4      for (j, peca2) in enumerate(pecas) #peça que será avaliada
5          for d1 in IFP[i]
6              for d2 in IFP[j]
7                  #verificações de overlapping com uso de inpolygon()
8              end
9              overlap = sort(overlap)
10             NFP[(i, j, d1)] = overlap
11         end
12     end
13 end
14

```

3.1.2 Modelagem com JuMP

O JuMP (*Julia for Mathematical Programming*) é um pacote de modelagem matemática desenvolvido para a linguagem Julia. Ele permite a escrita de modelos com sintaxe semelhante à notação matemática; o próprio pacote realiza a otimização integrando com algum *solver*, como CPLEX ou Gurobi ou algum outro.

Começamos definindo as variáveis de decisão: δ_t^d (variável de decisão binária, que indica a presença de uma peça do tipo t no ponto d) e z (variável inteira a ser minimizada).

```
1 @variable(model, delta[t in T, d in IFP[t]], Bin)
2 @variable(model, z >= 0, Int)
```

A seguir, estão as restrições do modelo, juntamente com um pequeno resumo de sua função no modelo, para facilitar o entendimento com relação a cada uma delas. Estas restrições já foram previamente apresentadas (respectivamente Equações (2)–(4)).

```
1 for t in T, d in IFP[t]
2     #deve ser grande o suficiente para caber
3     @constraint(model, (Int(floor((d - 1) / R)) * gx + maximum(x for (x,y) in pecas[t]))
4         ↪ * delta[t, d] <= z)
5
6 for t in T
7     #garante que todas as peças necessárias são colocadas
8     k = t
9     @constraint(model, sum(delta[k, d] for d in IFP[k]) == q[t])
10 end
11
12 for t in T, u in T, d in IFP[t], e in NFP[(t, u, d)]
13     #garante o não overlapping
14     if t != u && (e in IFP[u])
15         @constraint(model, delta[t, d] + delta[u, e] <= 1)
16     elseif q[t] > 1 && d < e && (e in IFP[t])
17         @constraint(model, delta[t, d] + delta[t, e] <= 1)
18     end
19 end
```

Por fim, é determinada a função objetivo do modelo (variável inteira z previamente definida), e realizada a otimização (função `optimize!()`) em si. Para visualizar o resultado ótimo e o tempo de execução, a biblioteca JuMP possui duas funções prontas (`objective_value()` e `solve_time()`), devidamente utilizadas.

```
1 @objective(model, Min, z)
2
3 optimize!(model)
4 println("Resultado ótimo: ", objective_value(model))
5 println("Tempo gasto: ", solve_time(model))
```

3.1.3 Função *plotting*

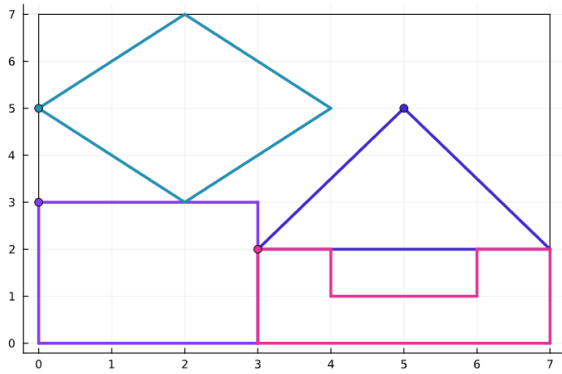
Dada a natureza geométrica do problema e o arranjo das peças, a visualização das soluções é crucial. Assim, foi desenvolvida uma função de *plot* para fornecer a visualização dos resultados encontrados pelo modelo. A função tem como parâmetros: o vetor de peças, o vetor de posições (tuplas que contêm o índice de posição d e o tipo de peça) além das informações de dimensão do *board*.

3.2 Resultados dos testes computacionais preliminares

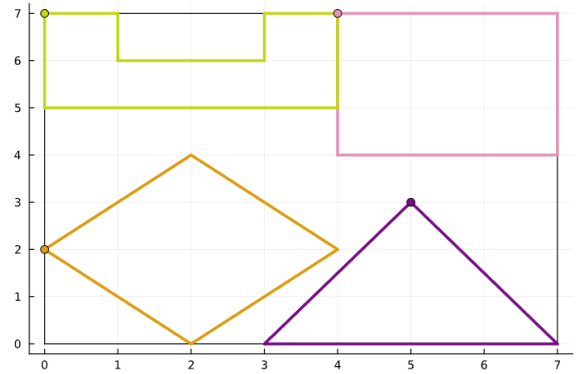
A seguir apresentamos, inicialmente, três exemplos-teste com o propósito de validar e ilustrar o funcionamento do método. Posteriormente, apresentamos testes realizados com instâncias da literatura, que apresentam maior escala e complexidade.

3.2.1 Primeiro exemplo

Para os primeiros testes, utilizamos o conjunto de peças já citado neste texto, na Seção 2.3, retirado de Toledo *et al.* [9]. Para o *board* foi utilizado 10 de comprimento e 7 de altura, que assegura que todas as peças caibam tranquilamente. Esta dimensão de *board* é mantida para os exemplos apresentados; em caso de alteração, será mencionado.



(a) Solução obtida pelo CPLEX.



(b) Solução obtida pelo Gurobi.

Figura 4: Resultado do primeiro exemplo.

Ambos os *solvers* chegaram no mesmo valor de solução ótima: comprimento $z = 7$. Entretanto, podemos observar na Figura 4 que os *solvers* obtiveram soluções diferentes. Em ambos os casos, o quadrado 3×3 e a peça não convexa ficaram lado a lado, assim como o losango e o triângulo.

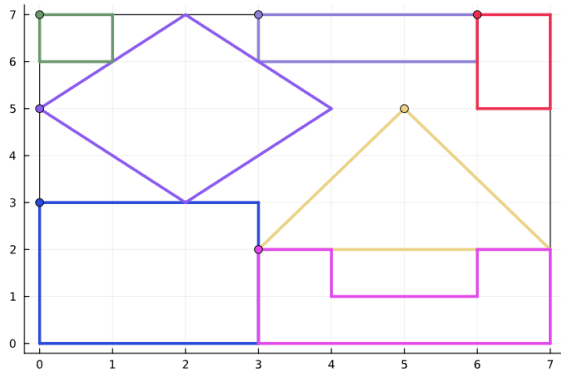
O tempo computacional para determinar cada solução foi o seguinte: 0,029 segundos pelo *solver* CPLEX e 0,048 segundos pelo *solver* Gurobi.

3.2.2 Segundo exemplo

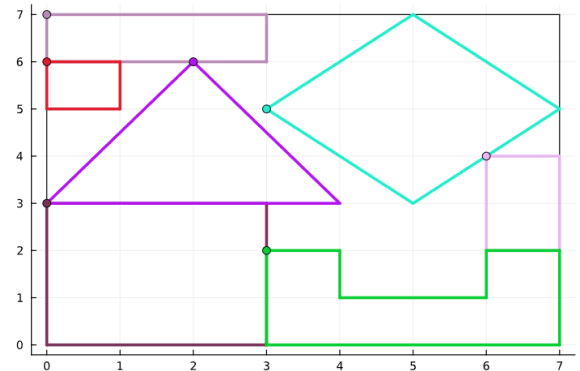
Para o próximo teste, adicionamos peças pequenas e simples com o intuito de testar a capacidade do modelo de tangenciar diferentes partes. Os polígonos adicionados foram:

- Quadrado 1×1 ;

- Retângulo 1×2 ;
- Retângulo 3×1 .



(a) Solução obtida pelo CPLEX.



(b) Solução obtida pelo Gurobi.

Figura 5: Resultado do segundo exemplo.

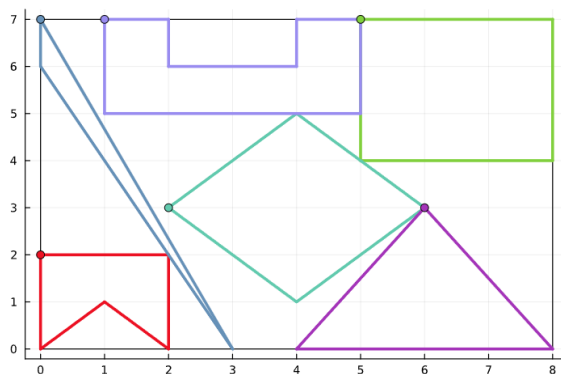
As três peças adicionadas, quando comparadas ao exemplo anterior, são pequenas e puderam ocupar espaços ociosos, sem alterar o valor da solução ótima, ou seja, o comprimento mínimo para alocar as sete peças continua igual a 7. Observando as soluções ótimas apresentadas na Figura 5, podemos notar que novamente os *solvers* determinaram soluções alternativas com o mesmo valor de função objetivo. (problema degenerado).

O tempo computacional para determinar cada solução foi o seguinte: 0,072 segundos pelo *solver* CPLEX e 0,130 segundos pelo *solver* Gurobi.

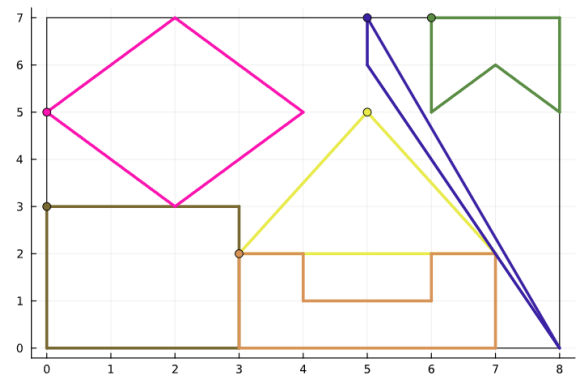
3.2.3 Terceiro exemplo

Agora adicionamos dois polígonos mais complexos para um impacto maior nos testes:

- Não convexo menor, se assemelha a uma bandeirola de festa junina;
- Triângulo escaleno com 7 de altura.



(a) Solução fornecida pelo CPLEX



(b) Solução fornecida pelo Gurobi

Figura 6: Resultado do terceiro exemplo.

A Figura 6 apresenta o resultado do terceiro exemplo. Observe que novamente os *solvers* determinam soluções ótimas alternativas. Observe, também, que para este conjunto de peças, o comprimento mínimo (ótimo) necessário para alocar todas as peças é 8.

O tempo computacional para determinar cada solução foi o seguinte: 0,115 segundos pelo *solver* CPLEX e 0,114 segundos pelo *solver* Gurobi.

Vale salientar que para os três exemplos apresentados, pudemos observar que o problema é degenerado, ou seja, apresenta soluções alternativas para o valor ótimo. Esta é uma característica muito comum em problemas de corte, dada sua natureza combinatória.

Enfim, com estes três exemplos, podemos constatar que o modelo está correto e resolve o problema proposto. Todas as restrições são respeitadas: não há sobreposições de peças, todas as peças estão dentro do *board* e o espaço utilizado é minimizado.

3.3 Dificuldades, correções e melhorias

Inicialmente, consideramos uma versão mais simples do problema, em que temos de alocar uma única peça de cada tipo, ou seja, $(q_i = 1, \forall i)$, conforme apresentamos nos três exemplos apresentados anteriormente. Essa escolha visou reduzir a complexidade computacional dos testes preliminares e ajudou na validação do modelo e a obtenção de resultados em menor tempo.

Em uma etapa posterior, o modelo foi estendido para incorporar múltiplas cópias por tipo de item. Essa generalização torna o problema mais aderente a aplicações reais, nas quais diferentes itens podem ter demandas variadas, e amplia significativamente o espaço de soluções viáveis, elevando o desafio da otimização.

Para esta adaptação, alteramos o conjunto de restrições (3). A reformulação consistiu na reatribuição direta do índice da variável, substituindo k por t , de modo que $k = t$ e na posterior utilização deste novo índice na variável $\delta_{t,d}$. Essa modificação viabilizou que o modelo tratasse adequadamente diferentes quantidades demandadas por item.

No trabalho de Toledo *et al.* [9], utilizado como base para este trabalho, não é detalhada a implementação do modelo, sendo assim, todos os trechos de código aqui apresentados são de autoria própria. Vale ressaltar que a implementação do $\mathcal{NFP}$ foi o trecho mais complexo e trabalhoso do projeto; foram testadas várias abordagens e métodos até chegarmos no que foi apresentado na Seção 3.1.1.

Uma etapa importante do modelo envolve a resolução do *grid*. De maneira intuitiva, quanto menor seu espaçamento – ou seja, quanto mais colunas e linhas são utilizadas (valores menores de g_x e g_y) – mais precisa será a aproximação das peças em relação às suas formas reais. No entanto, essa maior precisão tem um custo: o custo computacional aumenta drasticamente. Por essa razão, em todos os testes realizados, utilizamos g_x e g_y com o valor de 1.

Após a adaptação com relação ao conjunto de restrições (3) previamente mencionada, conforme sugerido em [9] mais dois conjuntos de restrições foram acrescentados ao modelo, com o intuito de refiná-lo e, possivelmente, diminuir os tempos de resolução.

O primeiro conjunto de restrições é dado por:

$$\delta_t^d + \delta_u^d \leq 1, \quad \forall t, u \in \mathcal{T} : (t, u) \in \mathcal{I}; \quad t \neq u; \quad \forall d \in \mathcal{D} \quad (7)$$

em que $\mathcal{I}$ é o conjunto de tipos incompatíveis, ou seja, peças desses tipos não podem ser posicionadas

no mesmo ponto. Observe que estas restrições avaliam sempre os dois δ s no mesmo ponto d . A seguir, apresentamos a implementação computacional:

```

1 for (t, u) in incompativeis, d in intersect(IFP[t], IFP[u])
2     @constraint(model, delta[t, d] + delta[u, d] <= 1)
3 end

```

Observe que, para facilitar sua implementação, dentro do código utilizado para o $\mathcal{NFP}$ (disponível em 3.1.2) foi construído o vetor de tipos incompatíveis, com uma simples verificação. A seguir, apresentamos o segundo conjunto de restrições adicionais:

$$\sum_{r \in \mathcal{R}} \delta_t^d \leq \lfloor \frac{W}{y_t^M - y_t^m} \rfloor, \quad \forall t \in \mathcal{T}; \quad \forall c \in \mathcal{C}; \quad d = c \times R + r + 1 \quad (8)$$

Este conjunto de restrições determina a quantidade de peças do mesmo tipo que podem ser "empilhadas", ou seja, alocadas na mesma coluna, dada a altura W do *board*.

4 Resultados finais

Nesta seção são apresentados resultados de testes realizados após as adições e correções mencionadas na Seção 3.3. Para tal, são utilizados os conjuntos de teste BLAZ, SHAPES e SHIRTS disponíveis em um repositório mantido por um grupo europeu de pesquisadores em problemas de corte e empacotamento, chamado ESICUP - *EURO Special Interest Group in Cutting and Packing* [4].

São apresentadas tabelas para comparação dos resultados obtidos nos testes com os resultados da literatura (Toledo *et al.* [9]) na coluna "Tempo [9]", quando disponível. Também nas tabelas, estão apresentados os limitantes superiores utilizados (valores iniciais de L) na coluna "Limitante", tempo para cada *solver* e o resultado ótimo (ou seja, o valor mínimo para z : comprimento utilizado). Além disso, apresentamos também uma comparação dos resultados sem e com as restrições adicionais citadas na Seção 3.3. Em casos de tempo de execução muito alto, estará escrito "TL" (*Time Limit*) nas colunas de tempo. Além disso, para algumas instâncias, não foram apresentados resultados em [9], nesses casos estará reportado "NI" (Não Informado) nas tabelas.

4.1 Conjunto BLAZ

Na Figura 7, a seguir, apresentamos as sete peças do conjunto de testes BLAZ:

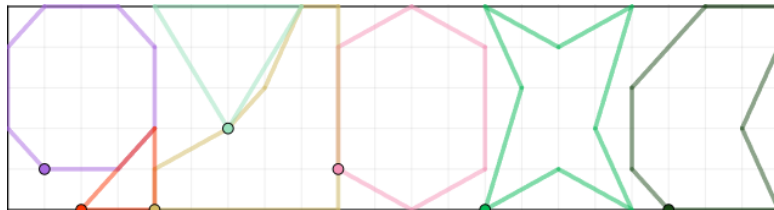


Figura 7: Conjunto de peças BLAZEWICS.

]

Os testes foram realizados utilizando $q_i = 4, i = 1, \dots, 7$ (denominado BLAZEIWCS4), ou seja, quatro cópias de cada uma das sete peças, totalizando 28 peças para esta instância-teste. O valor utilizado ($W = 15$) é padrão, fornecido pela documentação da instância. A seguir, as Tabelas 1 e 2 e a Figura 8 apresentam os resultados para esta instância.

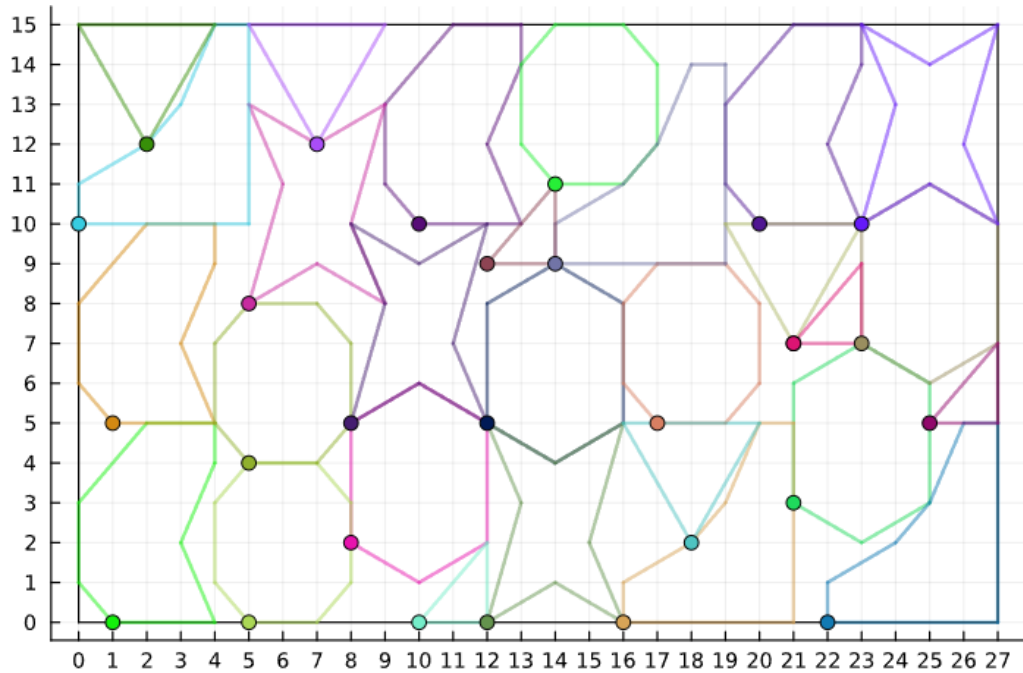


Figura 8: Possível disposição para resultado ótimo, conjunto BLAZ4.

Tabela 1: Resultados para BLAZEIWCS4.

Conjunto de peças	Limitante	Tempo em segundos		Tempo [9]	Resultado ótimo
		<i>CPLEX</i>	<i>Gurobi</i>		
BLAZEIWCS4	40	680,95	327,88	TL	27
	35	458,92	321,21		
	30	541,50	400,07		
	28	539,40	406,97		

Tabela 2: Resultados para BLAZEIWCS4 com restrições adicionais.

Conjunto de peças	Limitante	Tempo em segundos		Tempo [9]	Resultado ótimo
		<i>CPLEX</i>	<i>Gurobi</i>		
BLAZEIWCS4	40	1912,55	127,09	TL	27
	35	732,95	178,93		
	30	598,74	685,24		
	28	587,39	245,95		

Pelos resultados é possível perceber que, para este conjunto de peças o solver CPLEX não trabalhou bem com as restrições adicionais enquanto para o Gurobi foram ótimas adições. Além disso, é possível

notar que limitantes muito próximos do resultado ótimo nem sempre trarão resultados agradáveis, isso se deve ao caráter combinatório do problema, nem sempre o que se espera é o que realmente acontece.

4.2 Conjunto SHAPES

A Figura 9 apresenta os quatro tipos de peça do conjunto de testes SHAPES:

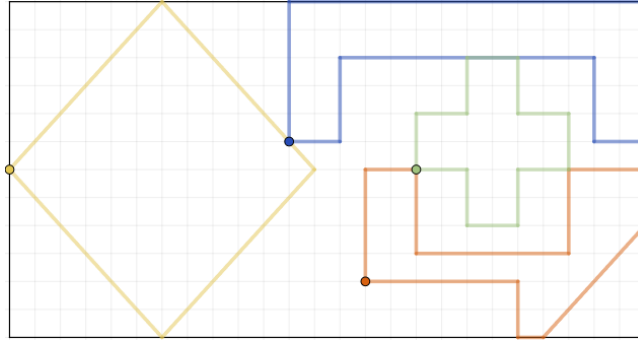


Figura 9: Conjunto de peças SHAPES.

Para este conjunto foram utilizadas duas instâncias teste, a primeira, mais reduzida, utilizando $q_i = 2, i = 1, \dots, 4$ (totalizando 8 peças, denominada SHAPES2), para a segunda considerou-se $q_i = 4, i = 1, \dots, 4$ (totalizando 16 peças, denominada SHAPES4). O valor utilizado ($W = 40$) é padrão, fornecido pela documentação da instância. A seguir, as Tabelas 3, 4, 5 e 6 e as Figuras 10 e 11 apresentam os resultados para esta instância.



Figura 10: Possível disposição para resultado ótimo, conjunto SHAPES2.

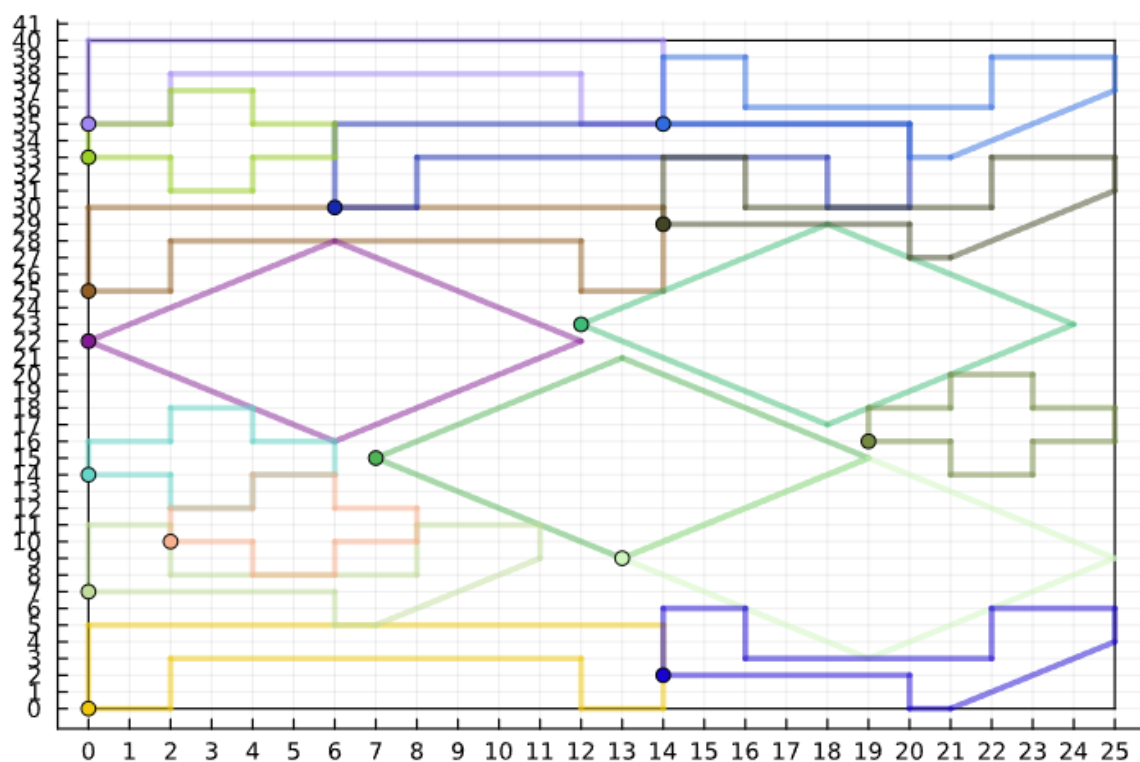


Figura 11: Possível disposição para resultado ótimo, conjunto SHAPES4.

Tabela 3: Resultados para SHAPES2.

Conjunto de peças	Limitante	Tempo em segundos		Tempo [9]	Valor ótimo
		<i>CPLEX</i>	<i>Gurobi</i>		
SHAPES2	20	6,97	2,24	NI	14
	18	6,50	1,23	NI	
	15	1,56	0,29	NI	
	14	0,63	0,28	0,45	

Tabela 4: Resultados para SHAPES4.

Conjunto de peças	Limitante	Tempo em segundos		Tempo [9]	Valor ótimo
		<i>CPLEX</i>	<i>Gurobi</i>		
SHAPES4	27	3.852,39	8.825,95	17.951,33	25
	25	2.520,19	5.812,80	NI	

Tabela 5: Resultados para SHAPES2 com restrições adicionais.

Conjunto de peças	Limitante	Tempo em segundos		Tempo [9]	Valor ótimo
		<i>CPLEX</i>	<i>Gurobi</i>		
SHAPES2	20	7,85	2,55	NI	14
	18	6,46	0,94	NI	
	15	1,68	0,51	NI	
	14	0,63	0,26	0,45	

Tabela 6: Resultados para SHAPES4 com restrições adicionais.

Conjunto de peças	Limitante	Tempo em segundos		Tempo [9]	Valor ótimo
		<i>CPLEX</i>	<i>Gurobi</i>		
SHAPES4	27	TL	8.016,92	17.951,33	25
	25	3.286,77	5.276,85	NI	

Novamente, para o CPLEX as restrições adicionais atrapalharam seu rendimento, para o conjunto SHAPES4 com limitante 27 o *solver* rendeu tão abaixo que atingia tempos na casa dos 30 mil segundos sem encontrar a solução ótima. Mas, ao contrário do conjunto BLAZ, aqui o Gurobi não se beneficiou tanto com a adição das restrições.

Observe que, para estas instâncias testes, os limitantes mais próximos do resultado ótimo apresentaram sim resultados melhores comparados aos limitantes mais “espaçados”.

4.3 Conjunto SHIRTS

Por fim, escolhemos um outro conjunto de testes da literatura para mostrar a utilidade prática do modelo, são moldes de uma indústria têxtil. Assim, para estas instâncias, realizamos os testes computacionais com o modelo em sua forma final, ou seja, com as restrições adicionais.

Este conjunto é chamado SHIRTS e tem oito tipos de peças que estão apresentadas na Figura 12.

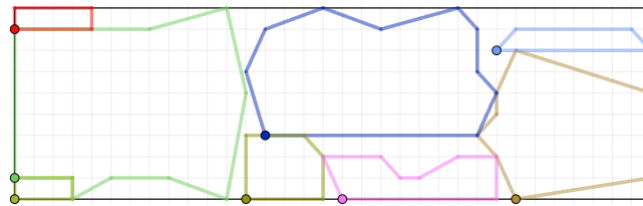


Figura 12: Conjunto de peças SHIRTS.

Os testes foram realizados utilizando $q_i = 2, i = 1, \dots, 8$ (totalizando 16 peças, denominado SHIRTS2). O valor utilizado ($W = 40$) é padrão, fornecido pela documentação da instância. A seguir, a a Figura 13 e a Tabela 7 apresentam os resultados para esta instância.

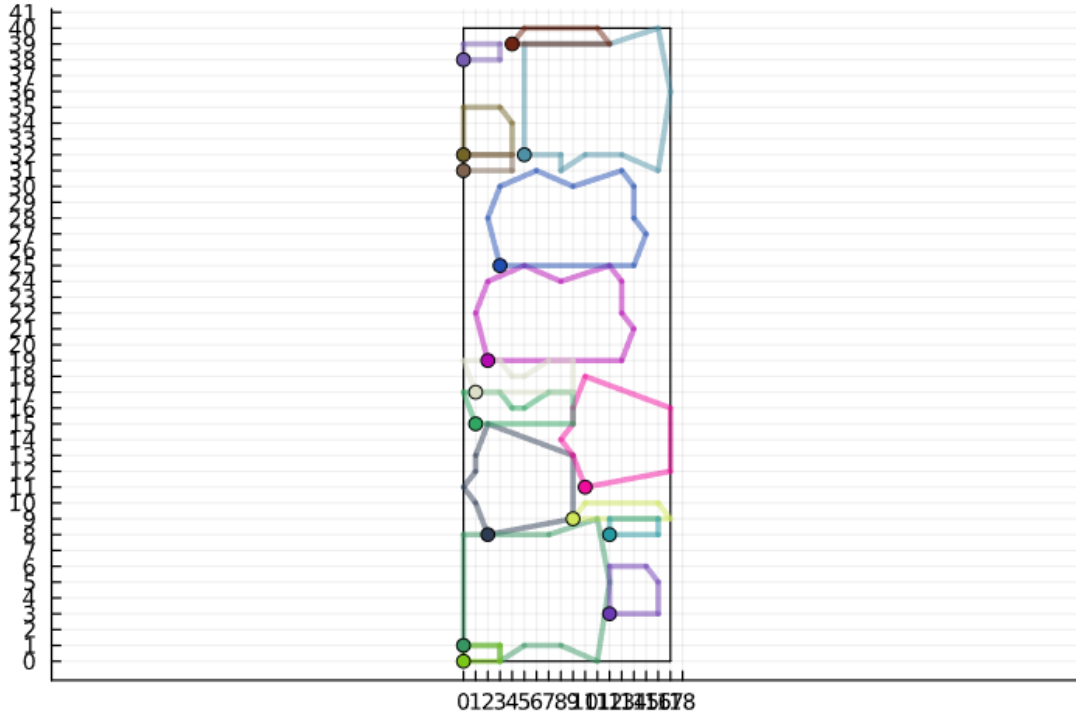


Figura 13: Possível disposição para resultado ótimo, conjunto SHIRTS2.

Tabela 7: Resultados para SHIRTS2 com restrições adicionais.

Conjunto de peças	Limitante	Tempo em segundos		Valor ótimo
		<i>CPLEX</i>	<i>Gurobi</i>	
SHIRTS2	20	70,85	15,60	17
	18	35,76	10,62	

Novamente, o solver Gurobi apresentou desempenho superior trabalhando com as restrições extras. Em ambos os *solvers* o limitante mais próximo apresentou desempenho superior.

5 Conclusão

Este trabalho apresentou com sucesso a implementação de um modelo de programação inteira para o problema de corte irregular (*nesting*), baseado na abordagem de discretização *dotted board model* [9]. A implementação, utilizando a linguagem Julia e a biblioteca JuMP, validou a eficácia do modelo ao resolver eficientemente instâncias da literatura, como BLAZ, SHAPES e SHIRTS, atingindo o objetivo esperado.

A análise comparativa demonstrou, em geral, um desempenho superior do solver Gurobi sobre o CPLEX, especialmente após o refinamento do modelo com restrições adicionais. O principal desafio computacional residu na complexa implementação do *No-Fit Polygon(NFP)*.

Concluimos que a abordagem discreta é uma forma viável e direta de abordar problemas na área de *nesting*. Os resultados obtidos foram satisfatórios, endossando a qualidade do modelo utilizado. Um possível avanço no trabalho é a considerar o objeto de formato irregular, o que implica em modelagem mais refinada.

Referências

- [1] Julia A. Bennell, Kathryn A. Dowsland e William B. Dowsland. “The irregular cutting-stock problem — a new procedure for deriving the no-fit polygon”. Em: *Computers & Operations Research* 28.3 (2001), pp. 271–287. ISSN: 0305-0548. DOI: [https://doi.org/10.1016/S0305-0548\(00\)00021-6](https://doi.org/10.1016/S0305-0548(00)00021-6). URL: <https://www.sciencedirect.com/science/article/pii/S0305054800000216>.
- [2] Julia A. Bennell e Jose F. Oliveira. “The geometry of nesting problems: A tutorial”. Em: *European Journal of Operational Research* 184.2 (2008), pp. 397–415. ISSN: 0377-2217. DOI: <https://doi.org/10.1016/j.ejor.2006.11.038>. URL: <https://www.sciencedirect.com/science/article/pii/S0377221706012379>.
- [3] Luiz H. Cherri et al. “Robust mixed-integer linear programming models for the irregular strip packing problem”. Em: *European Journal of Operational Research* 253.3 (2016), pp. 570–583. ISSN: 0377-2217. DOI: <https://doi.org/10.1016/j.ejor.2016.03.009>. URL: <https://www.sciencedirect.com/science/article/pii/S0377221716301370>.
- [4] EURO Special Interest Group in Cutting and Packing (ESICUP). *Data Sets*. <https://www.euro-online.org/websites/esicup/data-sets/>.
- [5] P. C. Gilmore e R. E. Gomory. “A linear programming approach to the cutting stock problem—part II”. Em: *Operations Research* 11.6 (1963), pp. 863–888.
- [6] P. C. Gilmore e R. E. Gomory. “A linear programming approach to the cutting-stock problem”. Em: *Operations Research* 9.6 (1961), pp. 849–859.
- [7] Antonio Miguel Gomes e José Fernando Oliveira. “A 2-exchange heuristic for nesting problems”. Em: *European Journal of Operational Research* 141.2 (2002), pp. 359–370. DOI: [https://doi.org/10.1016/S0377-2217\(02\)00130-3](https://doi.org/10.1016/S0377-2217(02)00130-3).
- [8] L. V. Kantorovich. “Mathematical methods of organizing and planning production”. Em: *Management Science* 6.4 (1960), pp. 366–422.
- [9] Franklina M.B. Toledo et al. “The Dotted-Board Model: A new MIP model for nesting irregular shapes”. Em: *International Journal of Production Economics* 145.2 (2013), pp. 478–487. ISSN: 0925-5273. DOI: <https://doi.org/10.1016/j.ijpe.2013.04.009>. URL: <https://www.sciencedirect.com/science/article/pii/S0925527313001722>.
- [10] Gerhard Wäscher, Heike Haußner e Holger Schumann. “An improved typology of cutting and packing problems”. Em: *European Journal of Operational Research* 183.3 (2007), pp. 1109–1130. ISSN: 0377-2217. DOI: <https://doi.org/10.1016/j.ejor.2005.12.047>. URL: <https://www.sciencedirect.com/science/article/pii/S037722170600292X>.